

# Quiz

- Go to **Quizzes > Quiz 09-15** on Canvas
- You have until **1:10pm** to complete it
- Allowed resources
  - Textbook
  - Your notes (on a computer or physical)
  - Course slides and website
- Resources not allowed
  - Generative AI
  - Internet searches

# Infini-gram: Scaling Unbounded n-gram Language Models to a Trillion Tokens

Jiacheng Liu<sup>1</sup>, Sewon Min<sup>1</sup>, Luke Zettlemoyer<sup>1</sup>, Yejin Choi<sup>1,2</sup>, Hannaneh Hajishirzi<sup>1,2</sup>

<sup>1</sup>University of Washington, <sup>2</sup>Allen Institute for AI

[\[Web Interface\]](#) [\[API Endpoint\]](#) [\[Python Package\]](#) [\[Docs\]](#) [\[Code\]](#) [\[Paper\]](#)

[🌟 **NEW**] Check out [infini-gram mini](#), a more storage-efficient index based on FM-index, with similar functionalities as infini-gram.

[🌟 **NEW**] Check out [OLMoTrace](#), an LLM behavior tracing tool we developed on top of infini-gram.

Join our [Discord server](#)! Get the latest updates & maintenance announcements, ask the developer anything about infini-gram, and connect with other fellow users.

It's year 2024, and n-gram LMs are making a comeback!!

We built an n-gram LM with the union of several open text corpora: {[Dolma](#), [RedPajama](#), [Pile](#), and [C4](#)}. The “n” in this n-gram LM can be arbitrarily large. This model is trained on **5 trillion tokens**, and contains n-gram counts for about 5 quadrillion (or 5 thousand trillion, or  $5 \times 10^{15}$ ) unique n-grams. It is **the biggest n-gram LM ever built to date**.

**BABE, MY LIFE IS A  
MARKOV CHAIN**

**GIVEN THE PRESENT, THE  
FUTURE DOES NOT  
DEPEND ON THE PAST**

# CS 1671 / CS 2071

## Human Language Technologies

### Session 7: N-gram language models

---

Michael Miller Yoder

September 15, 2026

# About Yuelyu Ji (TA)

- **4rd year IS PhD student**
- **Website:** <https://joydajunspacecraft.github.io/YuelyuJi.github.io>
- **Research interests:**
  - Natural language processing and information retrieval
  - Reliable AI agents and biomedical question answering
- **Office Hours:**
  - Thursday 5-6 pm
  - Zoom link in Canvas announcement
  - Email: yuj49@pitt.edu



# Assessments: project

- [Project idea form](#) to submit project ideas is **due today, Tue Sep 15**
- Take a look at the example projects on the [project website](#). You can submit one or more of those for the form, or submit your own idea!
- Have a potential project idea that involves deriving insight from a dataset of text, or building an NLP system that can do something with text? You can submit it!
  - Ideas do not need to be well-formed
  - Ideas that have data already available are more realistic
- You will later choose from an **anonymized** list of project ideas on Project Match Day, Sep 22

# Assessments: homework

- [Homework 1](#) is **due next Thu Feb 17**
- Covers text processing and regular expressions in Python

# Overview: N-gram language models

- Language modeling
- N-gram language models
- Sampling sentences from n-gram language models
- Estimating n-gram probabilities
- Perplexity and evaluating language models
- Handling zeros in n-gram language models
- Coding activity: build your own n-gram language model

# Structure of this course

## MODULE 1

### Prerequisite skills for NLP

text normalization, linear alg., prob., machine learning

## MODULE 2

statistical machine learning

n-grams

language modeling  
text classification

## MODULE 3

## MODULE 4

language modeling  
text classification  
sequence labeling

## MODULE 5

### NLP applications and ethics

machine translation, chatbots, information retrieval, bias



# Introduction to language models

---

# Language Models Estimate the Probability of Sequences

Which of these sentences would you be more likely to observe in an English corpus?

- Hugged I big brother my.
- I hugged my large brother.
- I hugged my big brother.



# Language Models Estimate the Probability of Sequences

Which of following word would be most likely to come after “David hates visiting New...”

- York
- California
- giggled



These are actually instances of the same problem: the language modeling problem!

# Language Modeling is Tremendously Useful

LMs (language models) are at the center of NLP today and have many different applications

- **Machine Translation**

$P(\text{high winds tonight}) > P(\text{large winds tonight})$

- **Spelling Correction**

$P(\text{about fifteen } \mathbf{minutes} \text{ from}) > P(\text{about fifteen } \mathbf{minuets} \text{ from})$

- **Text Input Methods**

$P(\text{i cant believe how hot you } \mathbf{are}) > P(\text{i cant believe how hot you art})$

- **Speech Recognition**

$P(\text{recognize speech}) > P(\text{wreck a nice beach})$

# The Goal of Language Modeling

Compute the probability of a sequence of words/tokens/characters:

$$P(\mathbf{w}) = P(w_1, w_2, w_3, w_5, \dots, w_n)$$

$$P(\text{I, hugged, my, big, brother})$$

This is related to next-word prediction:

$$P(w_t | w_1 w_2 \dots w_{t-1})$$

$$P(\text{York} | \text{David, hates, going, to, New})$$

Do you compute either of these? Then you're in luck:

**You are a language model!**

# N-gram language models

---

# The Chain Rule Helps Us Compute Joint Probabilities

The definition of conditional probability is

$$P(B|A) = \frac{P(A, B)}{P(A)}$$

which can be rewritten as

$$P(A, B) = P(A)P(B|A)$$

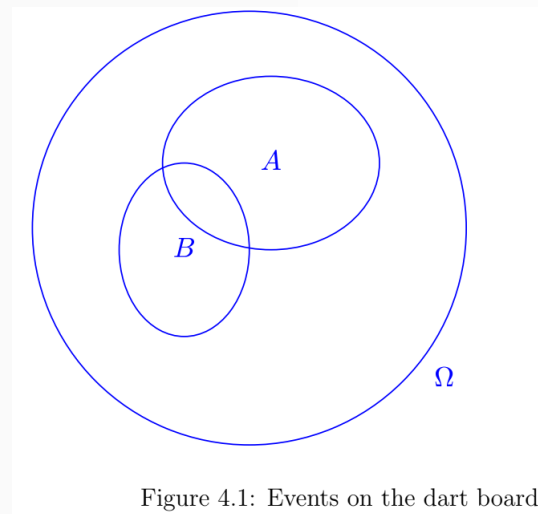


Figure 4.1: Events on the dart board



# The Chain Rule Helps Us Compute Joint Probabilities

If we add more variables, we see the following pattern:

$$P(A, B, C) = P(A)P(B|A)P(C|A, B)$$

$$P(A, B, C, D) = P(A)P(B|A)P(C|A, B)P(D|A, B, C)$$

which can be generalized as

$$P(x_1, x_2, x_3, \dots, x_n) = P(x_1)P(x_2|x_1)P(x_3|x_1, x_2) \dots P(x_n|x_1, \dots, x_{n-1})$$

**The Chain Rule!**

# The chain rule to compute the joint probability of words in a sentence

$$P(w_1, w_2, w_3, \dots, w_n) = \prod_i^n P(w_i | w_1 w_2 \dots w_{i-1})$$

$P(\text{now is the winter of our discontent}) =$   
 $P(\text{now}) \times P(\text{is}|\text{now}) \times$   
 $P(\text{the}|\text{now is}) \times P(\text{winter}|\text{now is the}) \times$   
 $P(\text{of}|\text{now is the winter}) \times$   
 $P(\text{our}|\text{now is the winter of}) \times$   
 $P(\text{discontent}|\text{now is the winter of our})$



# How Are We Estimating these Probabilities?

Could we just count and divide?

$$P(\text{discontent} | \text{now is the winter of our}) = \frac{\text{Count}(\text{now is the winter of our discontent})}{\text{Count}(\text{now is the winter of our})}$$

But this can't be a valid estimate! “now is the winter of our” is going to be very rare in corpora. It isn't going to be a good estimate of its true probability.

# This May not Seem Very Helpful

Is  $P(\text{discontent} | \text{now is the winter of our})$  really easier to compute than  $P(\text{now is the winter of our} | \text{discontent})$ ?

How can the chain rule help us? We can **cheat**.

# Markov Showed that You Could Make a Simplifying Assumption

Andrey Markov

One can approximate

$$P(\text{discontent}|\text{now is the winter of our})$$

by computing

$$P(\text{discontent}|\text{our})$$

or perhaps

$$P(\text{discontent}|\text{of our})$$



- We only get an estimate this way, but we can obtain it by only counting simpler things: “our discontent”, “discontent”, “of our”, etc
- N-gram language modeling is a generalization of this observation

# This assumption is the Markov assumption

$$P(w_1, w_2, \dots, w_n) \approx \prod_i P(w_i | w_{i-k} \dots w_{i-1})$$

In other words, we approximate each component in the product:

$$P(w_i | w_1, w_2, \dots, w_{i-1}) \approx P(w_i | w_{i-k} \dots w_{i-1})$$

We will now walk through what this looks like for different values of  $k$ .

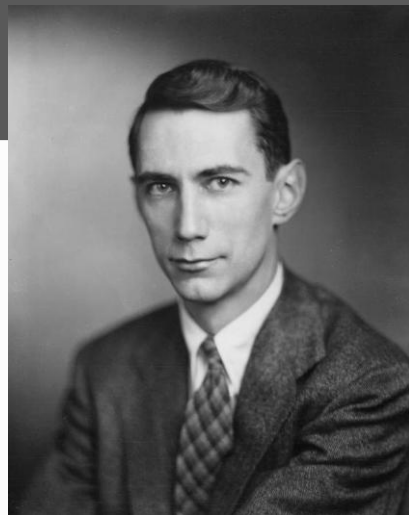
# Sampling sentences from language models

---

# The Shannon Visualization Method

- Choose a random bigram ( $\langle s \rangle$ ,  $w$ ) according to its probability
- Now choose a random bigram ( $w$ ,  $x$ ) according to its probability
- And so on until we choose  $\langle /s \rangle$
- Then string the words together

$\langle s \rangle$  I  
I want  
want to  
to eat  
eat Chinese  
Chinese food  
food  $\langle /s \rangle$   
I want to eat Chinese food





# Unigram model

$$P(w_1 w_2 \dots w_i) \approx \prod_i P(w_i)$$

The probability of a sequence is approximately the product of the probabilities of the individual words.

Some automatically generated sequences from a unigram model:

- fifth, an, of, futures, the, an, incorporated, a, a, the, inflation, most, dollars, quarter, in, is, mass
- thrift, did, eighty, said, hard, 'm, july, bullish
- that, or, limited, the

What do you notice about them?

# Bigram model

If you condition on the previous word, you get the following:

$$P(w_i | w_1 w_2 \dots w_{i-1}) \approx P(w_i | w_{i-1})$$

Some examples generated by a bigram model:

- texaco, rose, one, in, this, issue, is, pursuing, growth, in, a, boiler, house, said, mr., gurria, mexico, 's, motion, control, proposal, without, permission, from, five, hundred, fifty, five, yen
- outside, new, car, parking, lot, of, the, agreement, reached
- this, would, be, a, record, november

Are these better?

# Trigram model

The trigram model is just like the bigram model, only with a larger  $k$ :

$$P(w_i | w_1 w_2 \dots w_{i-1}) \approx P(w_i | w_{i-2} w_{i-1})$$

The output of a trigram language model is generally **much** better than that of a bigram model **provided the training corpus is large enough**. Why do you need a larger corpus to train a trigram corpus than a bigram or unigram corpus?

3  
gram

They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and Brazil on market conditions

# N-gram models have trouble with long-range dependencies

In general, n-gram models are very impoverished models of language. For example, language has relationships that span many words:

- The **students** who worked on the assignment for three hours straight **\*is/are** finally resting.
- The **teacher** who might have suddenly and abruptly met students **is/\*are** tall.
- Violins are easy to mistakenly think you can learn to play **\*them/quickly**.

# Ngram LMs Are Often Adequate

Nevertheless, for many applications, ngram models are good enough (and they're super fast and efficient)

# Estimating n-gram probabilities

---

# Estimating bigram probabilities with the maximum likelihood estimate (MLE)

MLE for bigram probabilities can be computed as:

$$P(w_i|w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$

which we will sometimes represent as

$$P(w_i|w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

# An example

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

$P(\text{I} | \text{<s>}) =$

$P(\text{Sam} | \text{<s>}) =$

$P(\text{am} | \text{I}) =$

$P(\text{</s>} | \text{Sam}) =$

$P(\text{Sam} | \text{am}) =$

$P(\text{do} | \text{I}) =$



# Bigram estimates of sentence probabilities

From a corpus, you could estimate probabilities of bigrams and then calculate probabilities of new sentences:

$P(< s> \text{ I want english food } < / s>) =$

$P(\text{I} | < s>)$

×  $P(\text{want} | \text{I})$

×  $P(\text{english} | \text{want})$

×  $P(\text{food} | \text{english})$

×  $P(< / s> | \text{food})$

$= .000031$

# Multiplication Considered Harmful

Doing computation in log space is preferred for language models

- **Avoid underflow** Multiplying small probabilities by small probabilities results in *very small* numbers, which is problematic
- **Optimize computation** Addition is cheaper than multiplication

$$\log(p_1 \times p_2 \times p_3 \times p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

# Perplexity and evaluating language models

---

# Perplexity is a metric for language modeling

Perplexity evaluates the probability assigned by a model **to a collection of test documents, controlling for length** and is, thus, useful for evaluating LMs.

A better model of a text is one which assigns a higher probability to words that actually occur in the test set. **Better models result in lower perplexities.**

However:

- It is a rather crude instrument
- It sometimes correlates only weakly with performance on downstream tasks
- It's only useful for pilot experiments
- But it's cheap and easy to compute, so it's important to understand



# Lower perplexity = better model

Training 38 million words, test 1.5 million words,  
WSJ

| N-gram<br>Order | Unigram | Bigram | Trigram |
|-----------------|---------|--------|---------|
| Perplexity      | 962     | 170    | 109     |

# The problem of zeros in n-gram language models

---

# The Perils of Overfitting

N-grams only work well for word prediction if the test corpus looks like the training corpus

- In real life, it often doesn't
- We need to train robust models that generalize!
  - One kind of generalization: Zeros!
  - Things that don't ever occur in the training set but occur in the test set

# N-grams in the test set that weren't in the training set

Suppose our bigram LM, trained on Twitter, reads a document by the philosopher Wittgenstein:

*Whereof one cannot speak, thereof one must be silent.*

This contains the bigrams: whereof one, one cannot, cannot speak, speak [comma], [comma] thereof, thereof one, one must, must be, be silent.

Suppose “whereof one” never occurs in the training corpus (**train**) but whereof occurs 20 times. According to MLE, it's probability is

$$P(\text{one}|\text{whereof}) = \frac{c(\text{whereof, one})}{c(\text{whereof})} = \frac{0}{20} = 0$$

The probability of the sentence is the **product** of the probabilities of the bigrams. What happens if one of the probabilities is zero?



# Strategies for handling zeros in n-gram LMs

- Laplace and Lidstone smoothing: simply add a small pseudocount to all possible n-grams in the vocabulary so none are 0
- There are more advanced methods that work better in practice, including interpolation and backoff

# Coding activity: build your own n-gram LM

---

# N-gram document representations on JupyterHub

1. Go to this [nbgitpuller link](#) (also available on course website)
2. Start a server with **TEACH – 6 CPUs - 45 GB**
3. Load custom environment at  
`/ix1/cs1671-2026f/class-env/.pixi/envs/default`
4. This should pull the `cs1671_fall2026_jupyterhub` folder into your JupyterLab
5. Open **session7\_ngram\_lm.ipynb**